COMP 3311
DATABASE MANAGEMENT SYSTEMS

LECTURE 24
TRANSACTION MANAGEMENT: RECOVERY SYSTEM; NOSQL TRANSACTIONS

DBMS FAILURE CLASSIFICATION

Transaction Failure

Logical errors: A transaction cannot complete due to some internal error condition (e.g., logic error, invalid input).

System errors: The database system must terminate an active transaction due to an error condition (e.g., deadlock).

System Crash

Due to a power, hardware or software failure.

Fail-stop assumption: we assume non-volatile (stable) storage contents are not corrupted by a system crash.

- E.g., database systems have numerous integrity checks to prevent corruption of secondary storage data.

Secondary Storage Failure

A disk failure (e.g., head crash) destroys all or part of disk storage.

Destruction is assumed to be detectable (e.g., disk drives use checksums to detect failures).

FAILURE RESPONSE

- To determine a DBMS's response to a failure we need to:

  a) identify failure modes of storage devices.

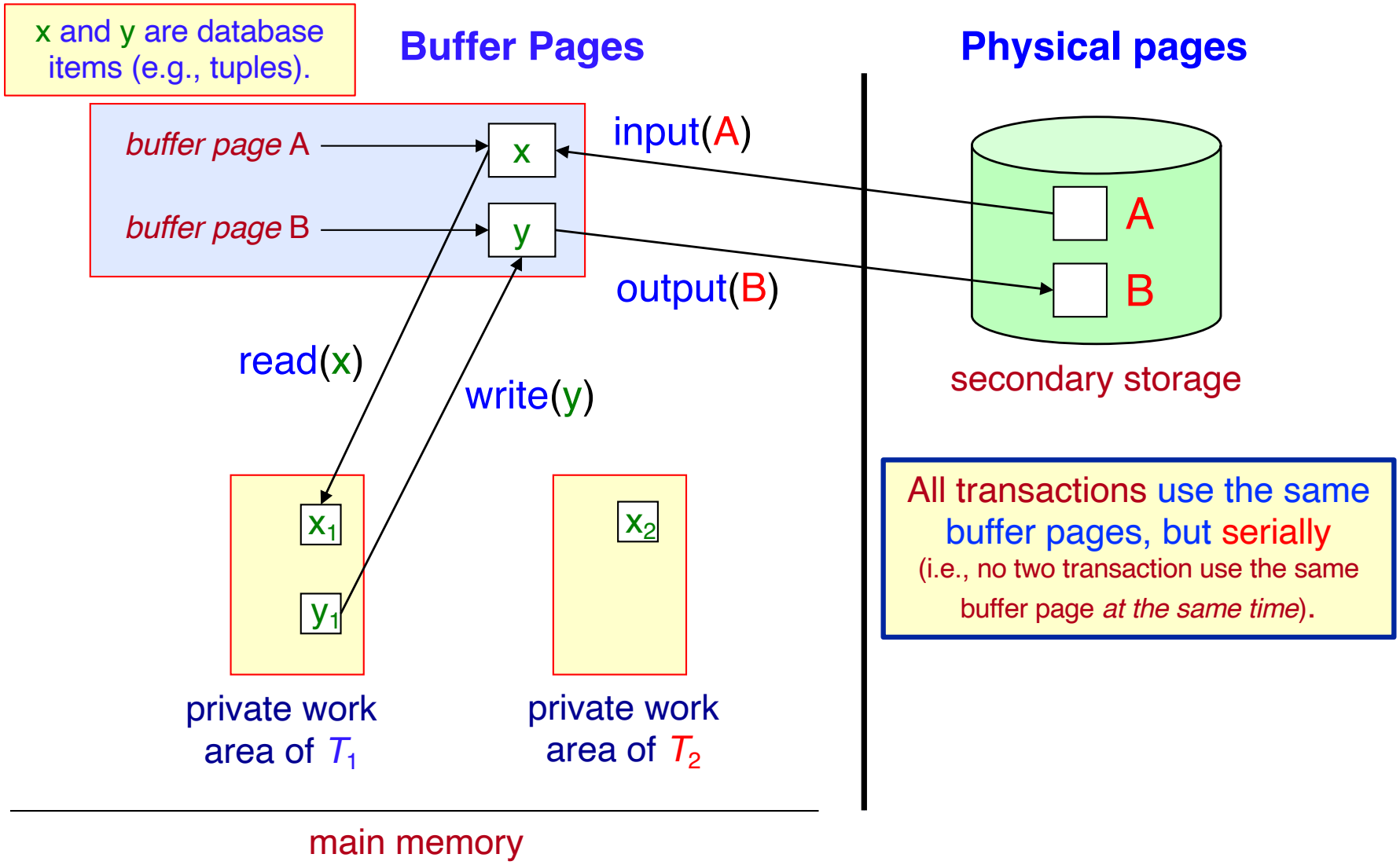
  b) consider how failure modes affect database content.

  c) develop **recovery algorithms** to ensure, despite failures,
     - transaction atomicity,
     - transaction durability and, consequently,
     - database consistency.

- Recovery algorithms have two parts:

  1. Actions taken during normal transaction processing to ensure enough information exists to recover from failures.
  2. Actions taken after a failure to recover the database contents to a state that ensures atomicity, durability and, thus, consistency.

DATA ACCESS ASSUMPTIONS



DATA ACCESS ASSUMPTIONS (CONT'D)

Physical pages – database pages residing on secondary storage.

Buffer pages – database pages residing temporarily in main memory.

- Page movements (secondary storage ⇔ main memory) use two operations:

  input(*B*) transfers physical page *B* to the buffer from secondary storage.

  output(*B*) transfers buffer page *B* to secondary storage replacing the physical page on secondary storage.

- Each transaction T_i has its private work area where local copies of all database items accessed and updated by it are kept (e.g., a variable in a program).

  T_i's local copy of a database item x is called x_i.

- We assume, for simplicity, that each database item fits in, and is stored inside, a single page.

DATA ACCESS ASSUMPTIONS (CONT'D)

- A transaction transfers database items between the buffer pages and its private work area using two instructions:

  read(x) assigns the value of database item x to the local variable x_i.

  write(x) assigns the value of local variable x_i to database item x in the buffer page.

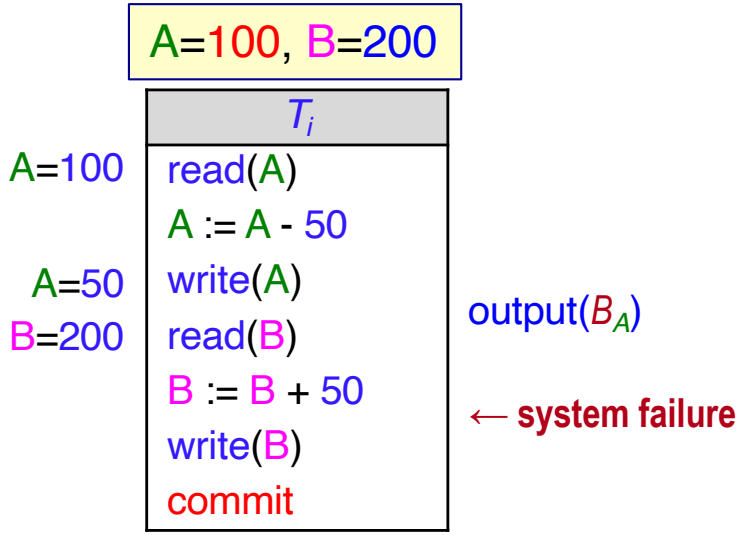
  ☞ **Both instructions may require an input(B_X), if the page B_X in which x resides is not already in the buffer.**

- Transactions
  - Perform a read(x) when accessing x for the first time.
  - All subsequent accesses are to the local copy x_i.
  - Perform write(x) after the last access to x_i.

  ☞ The operation output(B_X) does not need to immediately follow write(x) as the system can perform the output operations when it deems it necessary.

RECOVERY AND ATOMICITY

- Modifying the database without ensuring that the transaction will commit may leave the database in an inconsistent state.
- Several output operations may be required for T_i (e.g., to output the result of write(A) and write(B)).



- A failure may occur after some of these modifications (i.e., output operations) have been made, but *before* all of them are made.
- To ensure atomicity despite failures, we need to first output information describing the modifications made by a transaction to stable storage without modifying the database itself.

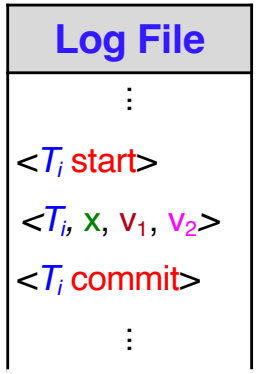
☞ **Our goal is to perform either *all* database modifications made by T_i or *none* of them.**

Assuming serial execution of transactions.

LOG-BASED RECOVERY

A log is a sequence of records that records all the update activities on the database.

☞ **A log is kept on stable storage (e.g., disk).**



- When transaction T_i starts, it registers itself by writing a <T_i start> log record.
- *Before* T_i executes write(x), it writes a <T_i, x, v_1, v_2> log record, where v_1 is the *value* of x *before* the write, and v_2 is the *new value* to be written to x.
  - The log record notes that T_i has performed a write on data item x; x had value v_1 before the write and will have value v_2 after the write.
- When T_i finishes its last instruction, it writes a <T_i commit> log record.
  - We assume, for now, that log records are written directly to stable storage (that is, they are not temporarily buffered in main memory).

Assuming serial execution of transactions.

DEFERRED DATABASE MODIFICATION

All modifications are recorded to the log, but all the writes to the database are deferred until after *partial commit*.

☞ **The old value is not needed in this scheme.**

- A transaction starts by writing a <T_i start> log record.
- A write(x) instruction writes a <T_i, x, v> log record, where v is the new value for x.
  - ☞ **The write of x *to the database* is not actually performed at this time but is deferred until later.**
- When T_i partially commits, a <T_i commit> log record is written.
- Finally, the log records are read and used to execute (i.e., *output to the database*) the previously deferred writes.
  - ☞ **Ensures recovery is possible after a failure.** Why?

Log File

⋮

<T_i start>

<T_i, x, v>

<T_i commit>

⋮

Assuming serial execution of transactions.

DEFERRED DATABASE MODIFICATION: RECOVERY

- During recovery after a crash, a transaction needs to be redone *if and only if* both <T_i start> and <T_i commit> are in the log (otherwise, log entries about T_i are ignored and T_i is re-executed).

- Redoing a transaction
  - redo(T_i) sets the value of all database items updated by the transaction to the new values (using the log information).

- Failures can occur while
  - the transaction is executing the original updates, or
  - during a recovery action.

Assuming serial execution of transactions.

DEFERRED DATABASE MODIFICATION: RECOVERY

Log File

The log below shows how it appears at three instances of time (a), (b) and (c).

<T_1 start>	<T_1 start>	<T_1 start>
<T_1, a, 950>	<T_1, a, 950>	<T_1, a, 950>
<T_1, b, 2050>	<T_1, b, 2050>	<T_1, b, 2050>
	<T_1 commit>	<T_1 commit>
	<T_2 start>	<T_2 start>
	<T_2, c, 600>	<T_2, c, 600>
		<T_2 commit>
Time (a)	Time (b)	Time (c)

Example

T_1 executes *before* T_2 with initial values a=1000, b=2000, c=700.

T_1	T_2
read(a)	
a := a - 50	
write(a)	
read(b)	
b := b + 50	
write(b)	
commit	
	read(c)
	c := c - 100
	write(c)
	commit

If the log on stable storage at the time of the failure is as in case:

(a) [after write(b)] No redo actions are needed since there is no commit for any transaction. Restart T_1.

(b) [after write(c)] Perform redo(T_1) since <T_1 commit> is present. Restart T_2.

(c) [after <T_2 commit>] Perform redo(T_1) followed by redo(T_2) since <T_1 commit> and <T_2 commit> are both present.

Assuming serial execution of transactions.

IMMEDIATE DATABASE MODIFICATION

Database updates of an uncommitted transaction can be made as its writes are issued.

☞ **Since undoing may be needed, the update log record must have both the old value and the new value.**

- The update log record must be written *before* the database item is written.
  - We assume that the log record is *output directly to stable storage*.
  - This can be extended to postpone log record output, so long as prior to performing an output(*B*) operation for a data page *B*, all log records corresponding to items in *B* have been output to stable storage.
- The output of updated pages can take place at any time before or after transaction commit.

☞ **The order in which pages are output to secondary storage can be different from the order in which they are written to in the buffer.**

Assuming serial execution of transactions.

IMMEDIATE DATABASE MODIFICATION: RECOVERY

- The recovery procedure has two operations instead of one:

  undo(T_i) restores the value of all database items updated by T_i to their old values, going backward from the last log record for T_i.

  redo(T_i) sets the value of all database items updated by T_i to the new values, going forward from the first log record for T_i.

- Both operations must be idempotent.
  - Even if the operation is executed multiple times the effect is the same as if it is executed only once.
  - This is needed since operations may get re-executed during recovery. Why?

- When recovering after failure:
  - Transaction T_i needs to be undone if the log contains the record $<T_i$ start$>$ but does not contain the record $<T_i$ commit$>$.
  - Transaction T_i needs to be redone if the log contains both the $<T_i$ start$>$ record and the $<T_i$ commit$>$ record.

☞ **Undo operations are performed first, then redo operations.**

Assuming serial execution of transactions.

IMMEDIATE DATABASE MODIFICATION: RECOVERY

Log File

The log below shows how it appears at three instances of time (a), (b) and (c).

Time (a)	Time (b)	Time (c)
<T_1 start>	<T_1 start>	<T_1 start>
<T_1, a, 1000, 950>	<T_1, a, 1000, 950>	<T_1, a, 1000, 950>
<T_1, b, 2000, 2050>	<T_1, b, 2000, 2050>	<T_1, b, 2000, 2050>
	<T_1 commit>	<T_1 commit>
	<T_2 start>	<T_2 start>
	<T_2, c, 700, 600>	<T_2, c, 700, 600>
		<T_2 commit>

Example

T_1 executes *before* T_2 with initial values a=1000, b=2000, c=700.

T_1	T_2
read(a)	
a := a - 50	
write(a)	
read(b)	
b := b + 50	
write(b)	
commit	
	read(c)
	c := c - 100
	write(c)
	commit

If the log on stable storage at the time of the failure is as in case:

(a) [after write(b)] undo(T_1): restore b to 2000 and a to 1000.

(b) [after write(c)] First undo(T_2) restoring c to 700. Then redo(T_1) setting a and b to 950 and 2050, respectively.

(c) [after <T_2 commit>] First redo(T_1) setting a and b to 950 and 2050, respectively. Then redo(T_2) setting c to 600.

CHECKPOINTS

Recovery Procedure Problems

1. Searching the entire log file is time-consuming.
2. We might unnecessarily redo transactions which have already output their updates to the database.

- We can streamline the recovery procedure by periodically performing checkpointing, which will
  1. output all log records currently residing in the buffer onto stable storage.
  2. output all modified buffer pages to secondary storage.
  3. write a <checkpoint> log record to the log file on stable storage.

☞ **Transactions cannot perform any update actions while a checkpoint is in progress.**

Assuming serial execution of transactions.

CHECKPOINTS (CONTD)

During recovery we need to consider only

i. the most recent transaction T_i that started *before* the checkpoint and *has not committed*.

ii. all transactions that *started after* the checkpoint.

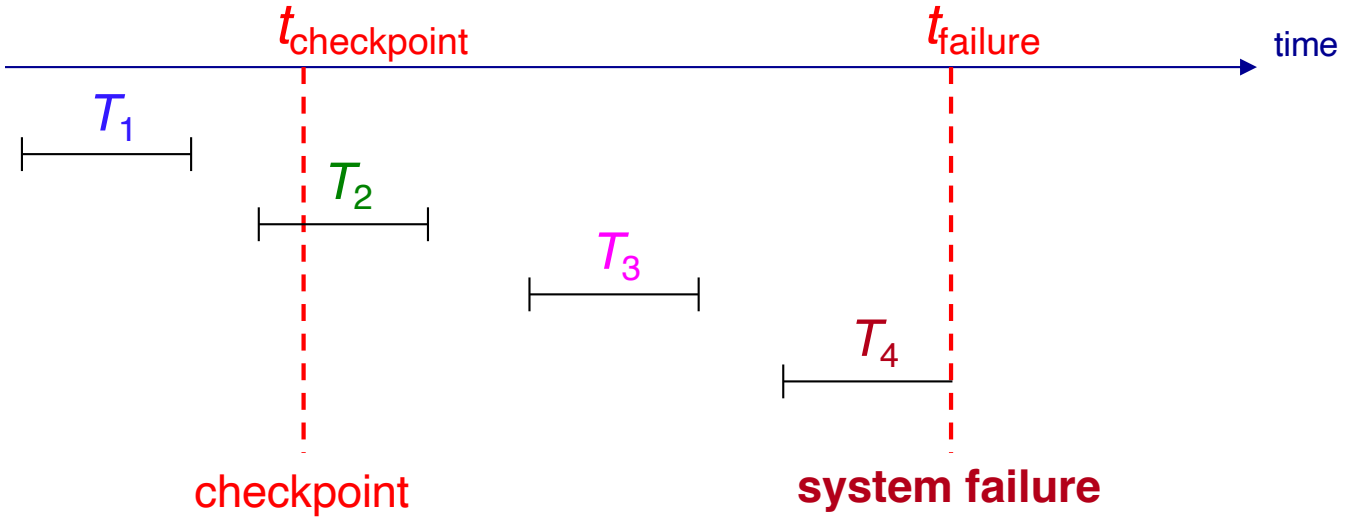
1. Scan backwards from the end of the log to find the most recent <checkpoint> record.
2. Continue scanning backwards until a <T_i start> record is found.
3. We need only consider the part of the log following this start record. The part of the log before this record can be ignored during recovery and can be erased whenever desired (*since we are assuming serial execution of transactions*).
4. Scan forward in the log starting from T_i.
   a) For all transactions with no <T_i commit> or <T_i abort> record, execute undo(T_i). (Done only when using immediate database modification protocol.)
   b) For all transactions with a <T_i commit> record, execute redo(T_i).

Assuming serial execution of transactions.

RECOVERY WITH CHECKPOINTS EXAMPLE

Log File

<T_1 start>
⋮
<T_1 commit>
<T_2 start>
⋮
<checkpoint>
⋮
<T_2 commit>
<T_3 start>
⋮
<T_3 commit>
<T_4 start>
⋮
system failure



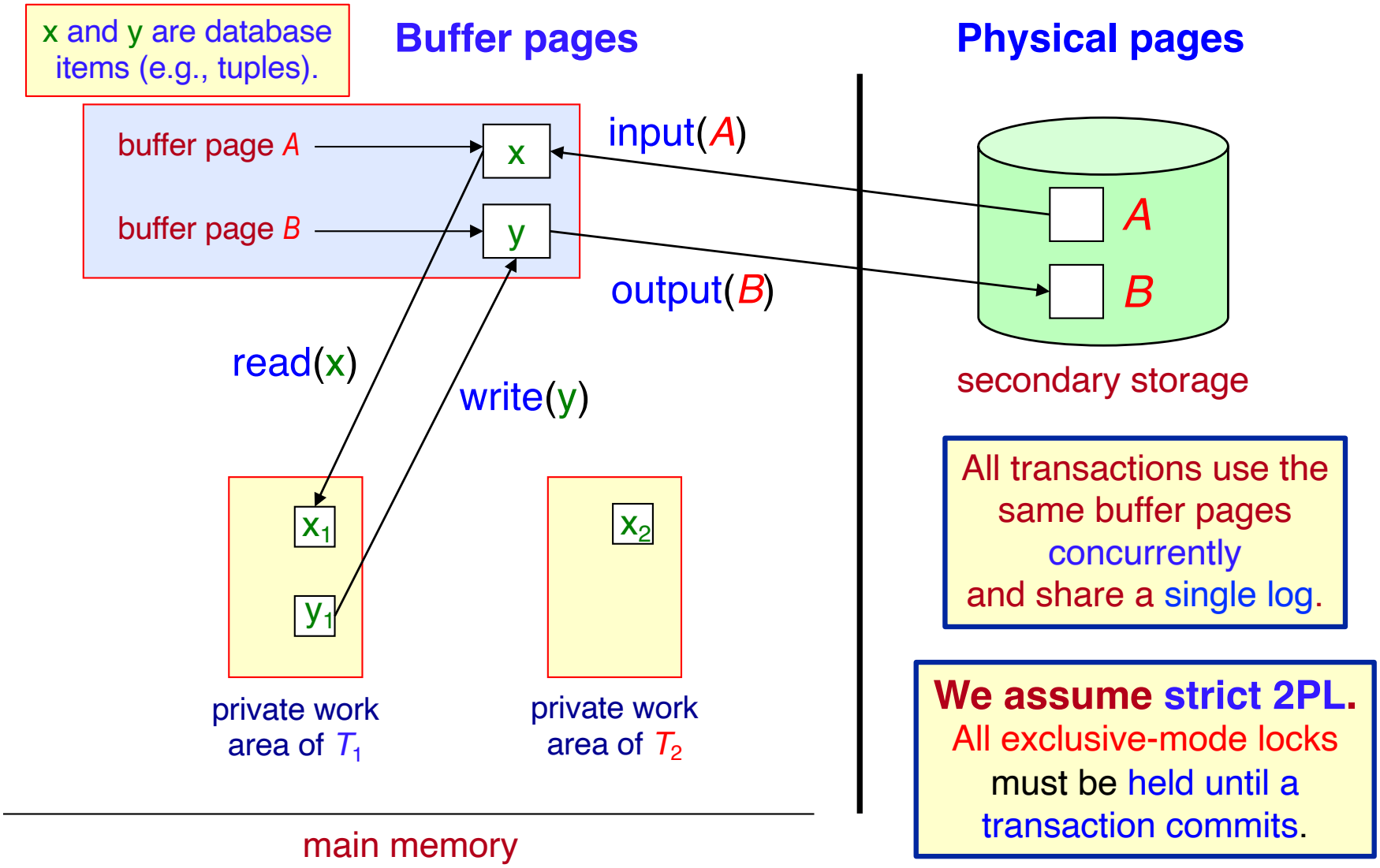
T_1 can be ignored since its updates were already output to secondary storage due to the checkpoint.

T_2 and T_3 need to be redone (since they have commit log records).

T_4 needs to be undone (since it has no commit log record).

1. Scan backwards from the end of the log to find the most recent <checkpoint> record.
2. Continue scanning backwards until a <T_i start> record is found.
3. Scan forward in the log starting from T_i.
   a) For all transactions with no <T_i commit> or <T_i abort> record, execute undo(T_i). (Done only in the case of immediate modification.)
   b) For all transactions with a <T_i commit> record, execute redo(T_i).

DATA ACCESS ASSUMPTIONS: RECOVERY WITH CONCURRENT TRANSACTIONS



All transactions use the same buffer pages concurrently and share a single log.

We assume strict 2PL.
All exclusive-mode locks must be held until a transaction commits.

RECOVERY WITH CONCURRENT TRANSACTIONS

- We modify the log-based recovery schemes to allow multiple transactions to execute *concurrently*.
  - All transactions share a single database buffer and a single log.
  - A buffer page can have database items updated by one or more transactions.
- We assume concurrency control using *strict two-phase locking*.
  - Recall that in strict 2PL a transaction holds all its exclusive locks until it commits – therefore, other transactions can only read "final" data, and the schedule is cascadeless.
- Logging is done as described earlier.
  - Log records of different transactions may be interspersed in the log.
- The checkpointing technique and actions taken on recovery must be changed since several transactions may be active when a checkpoint is performed.

RECOVERY WITH CONCURRENT TRANSACTIONS (CONT'D)

- The checkpoint log record is now of the form <checkpoint *L*> where *L* is the list of active transactions (i.e., uncommitted transactions) at the time of the checkpoint.

  ☞ **We assume no updates are in progress while the checkpoint is carried out.**

- When recovering from a failure, the system first does the following:
  1. Initialize *undo-list* and *redo-list* to empty.
  2. Scan the log backwards from the end, stopping when the first <checkpoint *L*> record is found.
  3. For each record found during the backward scan:
     i. If the record is <T_i commit>, add T_i to the *redo-list*.
     ii. If the record is <T_i start>, then if T_i is not in the *redo-list*, add T_i to the *undo-list*.
  4. For every T_i in *L*, if T_i is not in the *redo-list*, add T_i to the *undo-list*.

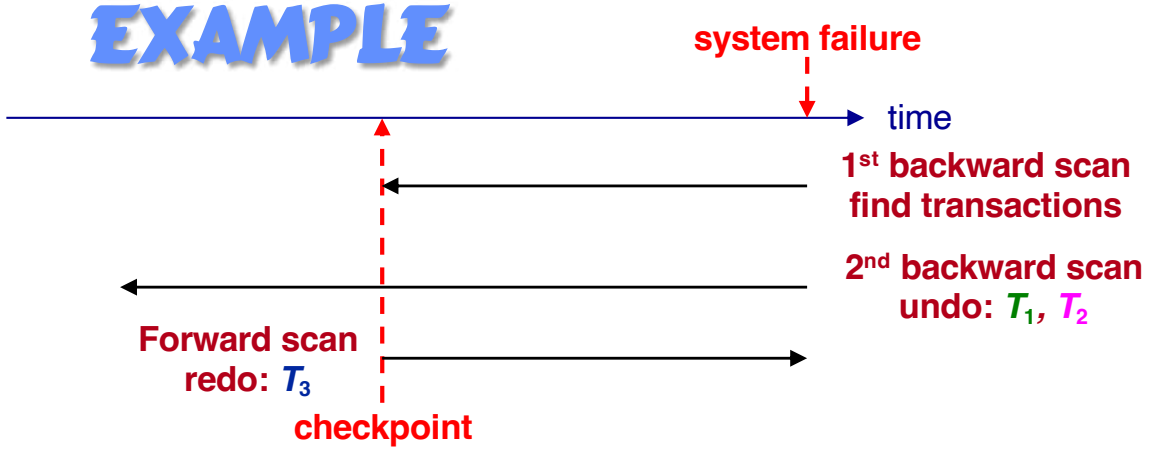
RECOVERY WITH CONCURRENT TRANSACTIONS (CONT'D)

- At this point, the *undo-list* consists of incomplete transactions which must be undone, and the *redo-list* consists of finished transactions that must be redone.
- Recovery now continues as follows:
  1. Scan the log backward from the most recent record, stopping when all the <T_i start> records have been encountered for every T_i in the *undo-list*.
     - During the scan, perform undo for each log record that belongs to a transaction in the *undo-list*.
  2. Locate the most recent <checkpoint *L*> record.
  3. Scan the log forward from the <checkpoint *L*> record until the end of the log.
     - During the scan, perform redo for each log record that belongs to a transaction in the *redo-list*.

☞ **Undoing backwards restores the original values, while re-doing forward sets each item to the most recent value.**

RECOVERY WITH CONCURRENT TRANSACTIONS EXAMPLE

Log File
<T_0 start>
<T_0, A, 0, 10>
<T_0 commit>
<T_1 start>
<T_1, B, 0, 10>
<T_2 start>
<T_2, C, 0, 10>
<T_2, C, 10, 20>
<checkpoint {T_1, T_2}>
<T_3 start>
<T_3, A, 10, 20>
<T_3, D, 0, 10>
<T_3 commit>
system failure



Undo:

Redo:

1st backward scan - find transactions:
1. If <T_i commit>, add T_i to the *redo-list*.
2. If <T_i start>, then if T_i is not in the *redo-list*, add T_i to the *undo-list*.
3. For every T_i in <checkpoint L>, if T_i is not in the *redo-list*, add T_i to the *undo-list*.

2nd backward scan - undo transactions:
1. Stop when all <T_i start> records for every T_i in the *undo-list* is found.
   - Perform undo for each transaction in the *undo-list*.

Forward scan - redo transactions:
1. Locate most recent <checkpoint L> record.
2. Perform redo for each transaction in the *redo-list*.

LOG RECORD BUFFERING

- Instead of writing log records individually, they can be buffered in main memory and output to stable storage when a page is full, or a log force operation is executed.

- A log force operation commits a transaction by forcing all its log records (including the commit record) to stable storage.

  ☞ **Several log records can be output by a single output operation.**

- The following rule, called the write-ahead logging or WAL rule, must be followed if log records are buffered in memory.

  1. Log records are output to stable storage in their create order.
  2. Transaction T_i enters the commit state only when the log record <T_i commit> has been output to stable storage.
  3. Before a page in the buffer is output to the database, all log records pertaining to data in that page must have been output to stable storage.

NOSQL DBMS TRANSACTIONS

- NoSQL DBMSs are distributed systems in which reads and writes may involve many nodes in the network.
- Consequently, transaction processing in NoSQL database systems must deal with transaction management issues arising from replication of data and failure of some nodes.
- Two types of transactions need to be considered:
  - local transactions that access and update data at only one node;
  - global transactions that access and update data at several nodes.
- To ensure the ACID properties
  - for local transactions, the protocols discussed so far can be used.
  - for global transactions, modification of these protocols is needed as well as additional protocols.

GLOBAL TRANSACTIONS

- Each site has a transaction coordinator that
  - starts the execution of transactions that originate at the site.
  - distributes subtransactions to appropriate sites for execution.
  - coordinates the termination of transactions that originate at the site.

- Each site also has a local transaction manager that
  - maintains a log for recovery purposes.
  - coordinates the execution and commit/abort of the transactions executing at that site.

- Commit protocols are used to ensure atomicity across sites.

  ☞ **A transaction which executes at multiple sites must either be committed at all the sites or aborted at all the sites.**

CONSISTENCY IN NOSQL DBMSS

- In a distributed database system, the network is likely to become partitioned at some point in time so that updates are no longer able to propagate to all nodes.

- Consequently, the database either becomes unavailable until the partitions are reconnected (as the updates cannot propagate to all nodes), or it becomes inconsistent (if some nodes get updated, but not others).

- Inconsistency is a non-starter for some applications (e.g., banking) but may not be a big problem for other applications (e.g., social networking).

- As availability is a major requirement for NoSQL DBMSs, the ACID properties, which focus heavily on consistency, may not be appropriate for these database systems.

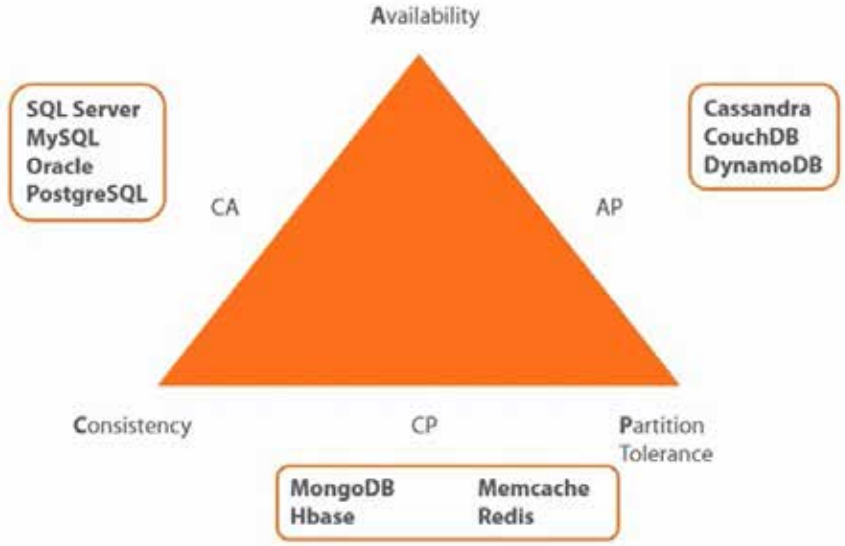
CAP THEOREM

A distributed database system cannot guarantee the following three properties *at the same time*:

Consistency	all nodes see the same data at the same time.
Availability	every request receives a response indicating a success or failure result.
Partition tolerance	the system continues to work even if some nodes go down.

- In the presence of a network partition, one must choose between consistency and availability.
  - Relational DBMSs choose consistency over availability.
  - NoSQL DBMSs choose availability over consistency.

Database systems and the CAP Theorem



EVENTUAL CONSISTENCY

- Rather than the ACID properties, NoSQL DBMSs follow the BASE principle.
  - Basically Available: NoSQL DBMSs adhere to the availability guarantee of the CAP theorem.
  - Soft state: the system can change over time, even without receiving input (due to delayed updates caused by the network).
  - Eventually consistent: the system will become consistent over time.

TRANSACTION MANAGEMENT: SUMMARY

- A transaction in relational DBMSs is required to have the ACID properties:
  - Atomicity, Consistency, Isolation, and Durability.

- Concurrency control protocols ensure that concurrent transaction execution is serializable and recoverable.

- The protocols differ in the way they handle conflicts:
  i. Lock-based protocols make transactions wait (thus, they can result in deadlocks).
  ii. Timestamp-based protocols make transactions abort (thus, there are no deadlocks, but aborting a transaction may be expensive).

- A recovery system ensures transaction atomicity, consistency and durability in the presence of system failures.

TRANSACTION MANAGEMENT: SUMMARY

- ACID properties can be enforced for transactions in NoSQL DBMSs.
  - But processing overhead may be high due to the need to coordinate among several sites.
  - And they may be unnecessary and/or unacceptable for some applications.
- BASE properties relax consistency requirements.
  - They are more appropriate for many web-based applications where availability and response time are more important than strict consistency.